

Migraciones de base de datos con Flyway

Spring + Tomcat. Sin magia.

← → navegar · T tema · Ctrl+P a PDF

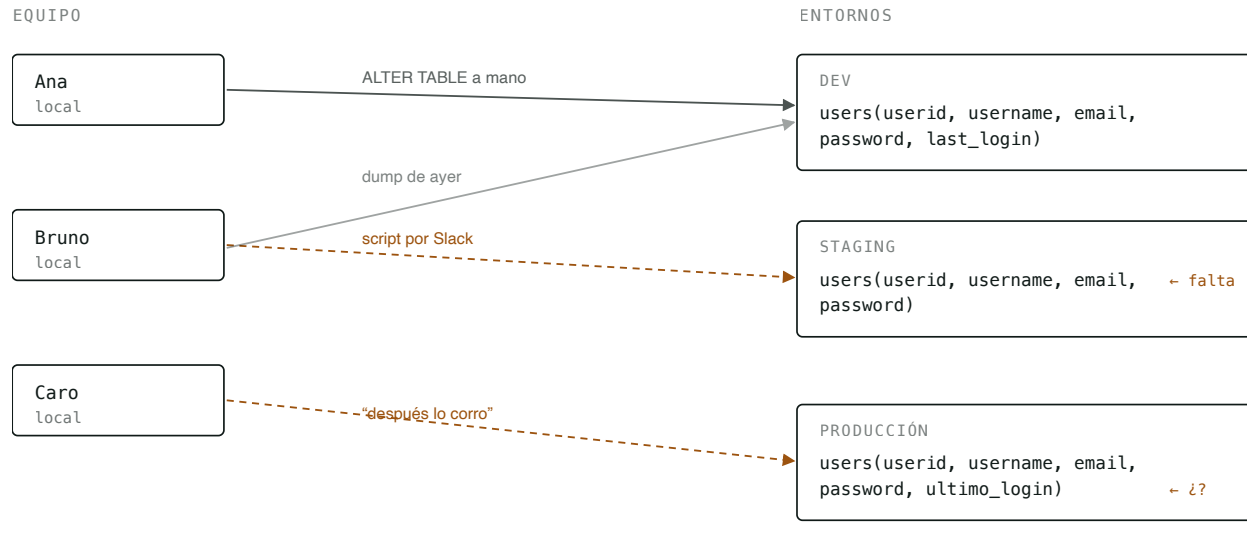
PUNTO DE PARTIDA

LA APP ARRANCÓ · PSQL → \D USERS

```
userid   | integer      | not null  
username | varchar(255) | not null  
email    | varchar(255) | not null  
password | varchar(255) | not null
```

Anda. Ahora quiero guardar cuándo fue el último login.
¿Cómo le agrego una columna a esta tabla?

“Le corro un **ALTER TABLE** y listo”



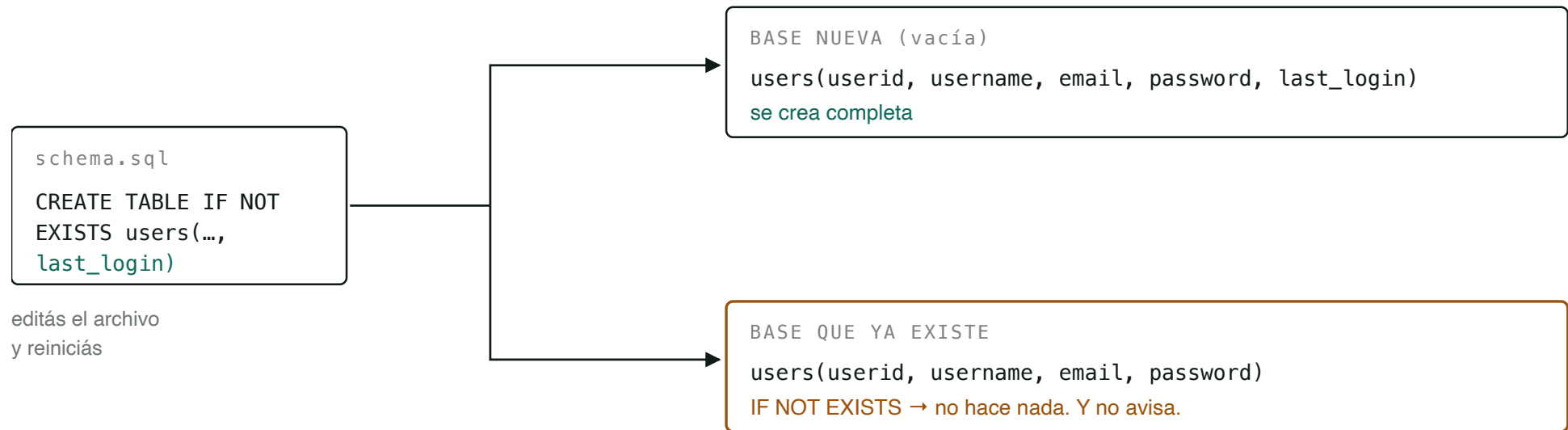
Tres entornos, tres esquemas distintos.

El bug de producción no se reproduce en dev, porque dev no es como producción.

Cada flecha es una decisión humana que alguien puede olvidar, repetir, o hacer distinto.

“Lo pongo en `schema.sql`, que corre solo al arrancar”

Mejor: ahora está en Git, lo corre la máquina y nadie se lo olvida. Pero tiene un techo.



El mismo archivo, dos resultados distintos. `CREATE TABLE IF NOT EXISTS` sirve para crear, nunca para modificar.

¿Y si le agrego los `ALTER` abajo, en el mismo archivo? Funciona — hasta que aparece una sentencia sin `IF NOT EXISTS`. Esta, en el segundo arranque, deja la app sin levantar:

```
ALTER TABLE users ADD CONSTRAINT users_email_unique UNIQUE (email);
-- 2do arranque → ERROR: relation "users_email_unique" already exists
```

Flyway versiona el esquema de tu base

1. Busca archivos `.sql` en una carpeta del proyecto.
2. Se fija, en la propia base, cuáles ya se aplicaron.
3. Corre en orden únicamente los que faltan.

No es un ORM y no genera SQL por vos. El SQL lo escribís vos; Flyway se encarga de que corra **una vez, en orden, en todas las bases**.

Escribiendo una migración

Un archivo `.sql`, en la carpeta que Flyway mira.

PERSISTENCE/SRC/MAIN/RESOURCES/DB/MIGRATION/

```
V1__Create_users_table.sql
```

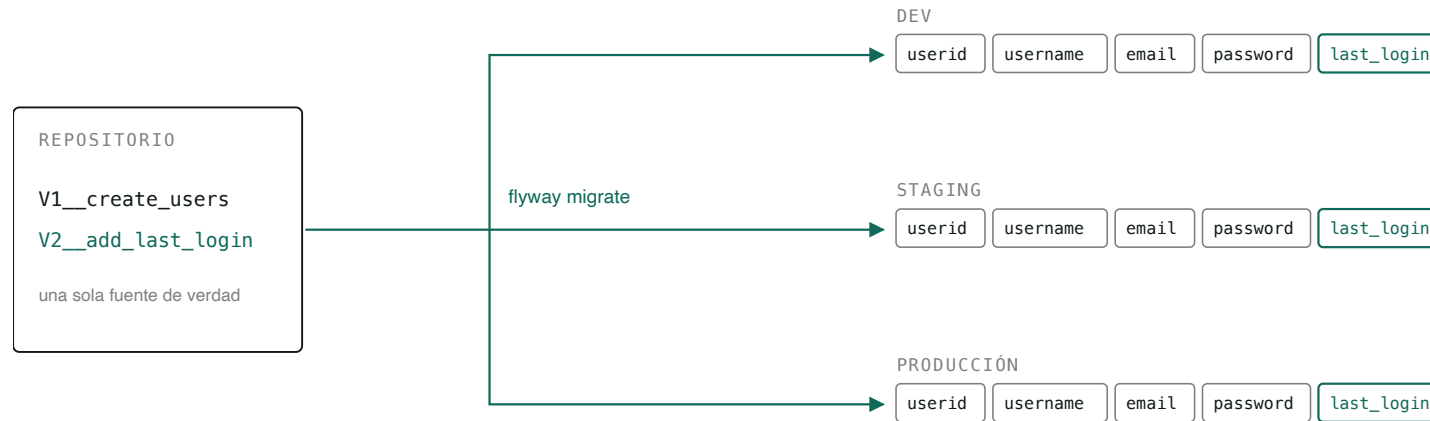
SU CONTENIDO

```
CREATE TABLE users (  
    userId    SERIAL PRIMARY KEY,  
    username  VARCHAR(255) NOT NULL UNIQUE,  
    email     VARCHAR(255) NOT NULL UNIQUE,  
    password  VARCHAR(255) NOT NULL  
);
```

Eso es todo. Cuando arranca la aplicación, Flyway encuentra el archivo, ve que esta base todavía no lo tiene aplicado, y lo ejecuta.

El nombre no es decorativo — el `V1` es la versión, y de ahí sale el orden. Lo miramos en detalle más adelante.

Una secuencia de cambios, no un estado final



Mismos archivos, mismo orden, mismo esquema. En los tres.

Cada cambio es un archivo propio, con su autor y su PR — y cada base sabe hasta dónde llegó.

Una migración es un `.sql` con el nombre bien puesto

`V2__Add_last_login_to_users.sql`

V = versionada

versión (orden)

DOS guiones bajos

descripción legible

siempre .sql

```
ALTER TABLE users ADD COLUMN last_login TIMESTAMP;
```

V – VERSIONADA

Corre **una sola vez**, en orden. Es el 95 % de lo que vas a escribir.

R – REPETIBLE

`R__nombre.sql`, sin número. Corre cuando cambia su contenido.
Para vistas y funciones.

El error de la primera clase: escribir `V4_Add_last_login.sql` con **un** guión bajo. Flyway ignora el archivo sin decir nada.

Cómo sabe Flyway qué ya corrió

Lo anota en una tabla, en tu propia base. La crea sola.

VERSION	DESCRIPTION	CHECKSUM	INSTALLED_ON	SUCCESS
1	Create users table	1938472615	2026-08-30 14:02:11	t

version

Todo archivo con versión mayor está pendiente.

checksum

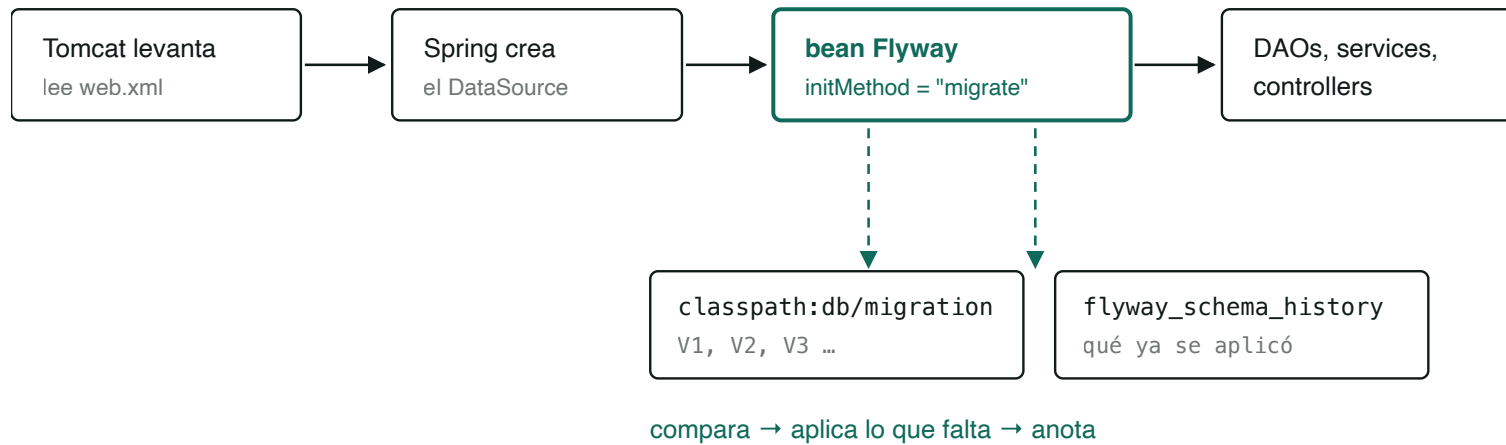
Hash del archivo. Si cambia después de aplicado, Flyway frena.

success

Si una migración falló, queda en **false**.

Una base vacía no tiene esta tabla, así que no sabe nada — y Flyway le aplica todo desde V1.

Qué pasa cuando arranca Tomcat



Si una migración falla acá, la aplicación no levanta. Es a propósito.

Flyway tiene que terminar *antes* de que cualquier DAO toque el esquema.

La dependencia de Flyway

```
<flyway.version>9.22.3</flyway.version>
```

```
<dependency>
```

```
  <groupId>org.flywaydb</groupId>
```

```
  <artifactId>flyway-core</artifactId>
```

```
  <version>${flyway.version}</version>
```

```
</dependency>
```

Un solo artefacto — `flyway-core` — alcanza para todo lo anterior.

El cableado

WEBAPP/.../CONFIG/WEBCONFIG.JAVA

```
@Bean(initMethod = "migrate")
public Flyway flyway(DataSource dataSource) {
    return Flyway.configure()
        .dataSource(dataSource)
        .locations("classpath:db/migration")
        .load();
}
```

DÓNDE VIVEN LOS ARCHIVOS

```
persistence/src/main/resources/db/migration/
└─ V1__Create_users_table.sql ← por ahora, una sola
```

`initMethod = "migrate"`

Spring llama `migrate()` al crear el bean, durante el arranque del contexto.

`DataSource` por parámetro

Garantiza que exista *antes* que Flyway. El orden deja de ser casualidad.

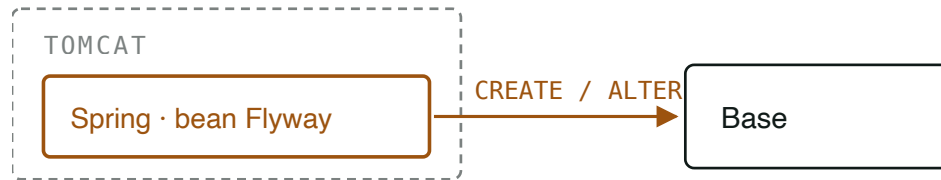
Migraciones en `persistence`

El módulo que sabe de tablas es el que las define. Viajan en el jar, dentro del war.

Los tests de `persistence` corren **estas mismas migraciones** contra HSQLDB en memoria. Si una migración rompe, se entera el test antes que producción.

¿Quién corre las migraciones en producción?

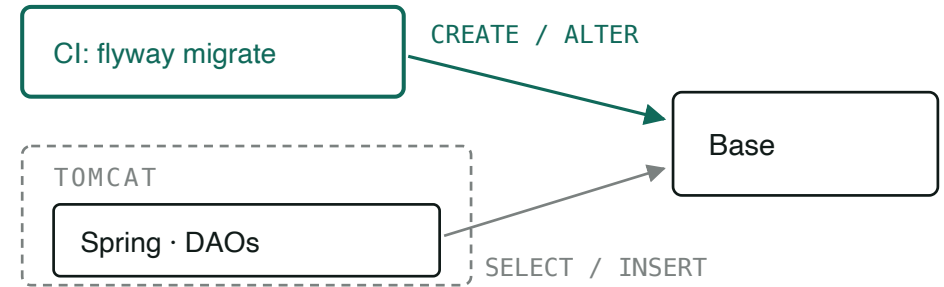
A · MIGRA LA APLICACIÓN



Migra Spring, no Tomcat: Tomcat solo lo hospeda.
Esa app necesita permisos de DDL en producción.

local · desarrollo · esta materia

B · MIGRA EL PIPELINE



El DDL lo corre el pipeline, una sola vez.
Spring arranca sin tocar el esquema.

staging · producción

“La app” acá es el bean de Flyway dentro del contexto de Spring — Tomcat es solo el contenedor que lo hospeda. La diferencia entre A y B es una sola: **quién tiene permisos de DDL sobre la base**. Se usan los dos, y los archivos `.sql` son exactamente los mismos.

Las cuatro reglas que no se rompen

1

Una migración aplicada es inmutable

Nunca edites un archivo que ya corrió. Cambia el checksum y Flyway se planta.

2

Se arregla hacia adelante

¿Te equivocaste en V3? No lo toques: escribí V4. La historia es un log, no un borrador.

3

No hay "deshacer"

Las migraciones `U__` son de la edición paga. Revertir es escribir otra migración.

4

Un solo dueño del esquema

Si migra Flyway, no migra nadie más. Ni `schema.sql`, ni Hibernate, ni vos con `psql`.